



**Universidad Nacional
Autónoma de México**



Facultad de Ingeniería

**Laboratorio de Arquitectura y Organización
de Computadoras**

Grupo: 07 - Semestre: 2027-1

Reporte Practica 3

Construcción de Máquinas de estados Usando Memorias
Direccionamiento por Trayectoria

Fecha de Entrega

02 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Familiarizar al alumno en el conocimiento de construcción de máquinas de estados usando memorias con el método de direccionamiento por trayectoria.

2. Desarrollo

Se implementó el circuito digital de una lavadora industrial mediante una máquina de estados finitos implementada con una memoria ROM. El funcionamiento se basa en tres entradas: el botón de inicio X , el sensor de tapa cerrada Y y el sensor de agua llena Z . El sistema cuenta con cuatro salidas: la válvula de llenado de agua V , la bomba de drenaje B , el indicador de alarma P y la señal de control del motor M . Es importante considerar que las salidas, V y B son salidas en lógica positiva, mientras que P y M se expresan con lógica negada.

El diseño fue construido implementado un método de direccionamiento por trayectoria. Donde el registro conserva el estado presente, mientras un bloque de concatenación une dicho estado con las entradas para formar la dirección de la ROM, y la palabra leída se divide en liga (estado siguiente) y salidas. A partir de esta implementación, cada trayectoria de la carta ASM queda codificada directamente en una localidad de memoria, evitando que sea necesario realizar simplificaciones mediante mapas de Karnaugh.

2.1. Carta ASM

La Figura[1] corresponde al Circuito Digital de la Lavadora Industrial anteriormente descrito.

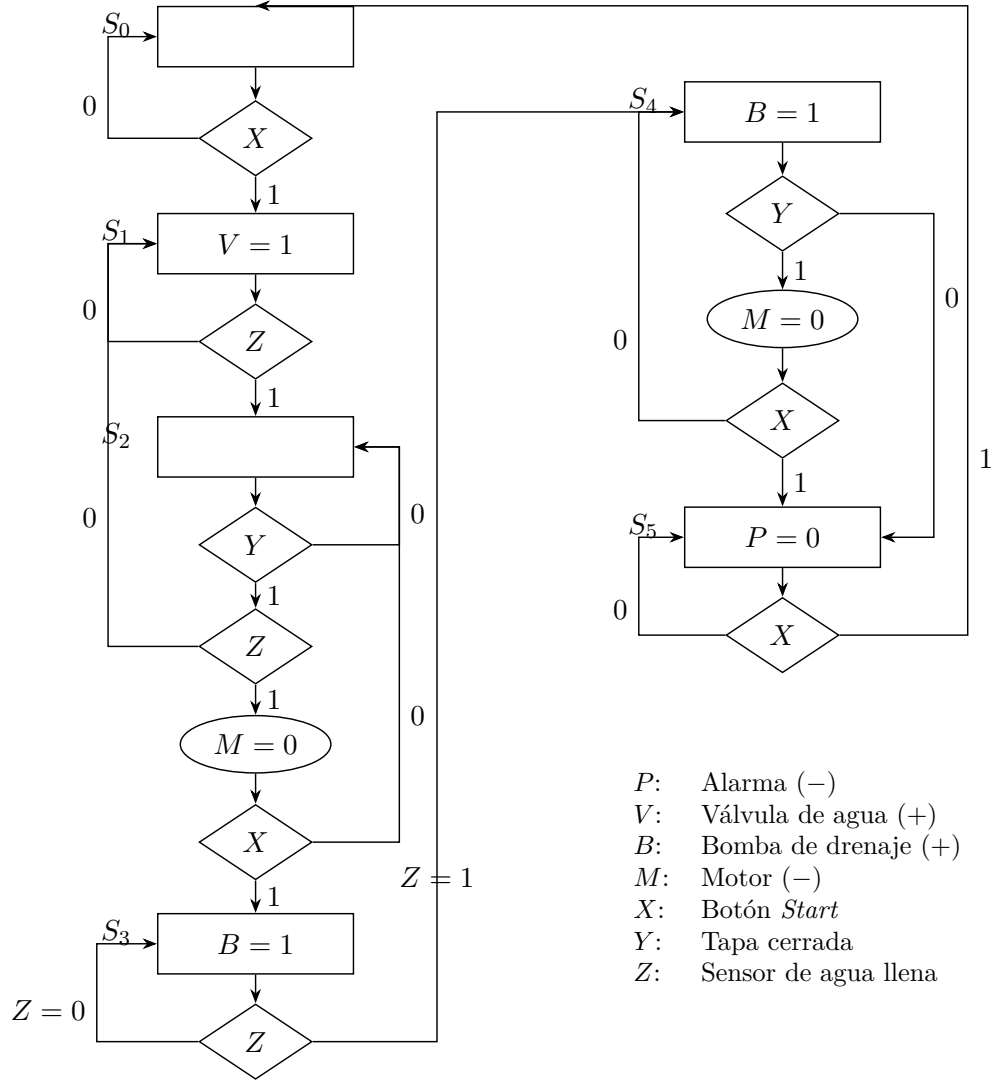


Figura 1: Carta ASM de la Lavadora industrial.

2.2. Dimensionamiento de la memoria

Para implementar la carta ASM mediante direccionamiento por trayectoria, se requiere una memoria ROM que almacene las ligas y las salidas correspondientes a cada combinación de estado presente y entradas.

Se considera que la máquina tiene 6 estados (S_0 a S_5), tres entradas y cuatro salidas. Se requieren 3 bits ($q = 3$) para codificar el estado. En el direccionamiento por trayectoria, la

dirección combina el estado presente con las entradas (I), la palabra de memoria se compone de la liga y las salidas (O). De forma que los bits de dirección y de palabra se calculan como:

$$\underbrace{n}_{\text{bits de dirección}} = q + I = 3 + 3 = 6, \quad \underbrace{m}_{\text{bits por palabra}} = q + O = 3 + 4 = 7.$$

La ROM requerida es de $2^n \times m = 2^6 \times 7 = 64 \times 7$ bits, con una capacidad total de 448 bits. La dirección se organiza como Q2 Q1 Q0 X Y Z y el contenido de cada palabra como L2 L1 L0 V B P M.

2.3. Tabla de estados, entradas y salidas

Para construir la tabla de direccionamiento, se recorrió la carta ASM y se registraron las combinaciones de estado presente, entradas, liga y salidas. La Tabla[1] muestra los resultados obtenidos. Las columnas *liga* y *salidas* se escriben en el mismo orden usado en la ROM: L2L1L0 y VBPM, respectivamente.

Tabla 1: Tabla de transición y salidas obtenida de la carta ASM.

Estado	$Q_2Q_1Q_0$	X	Y	Z	Liga	Salidas
S_0	000	0	*	*	000	0011
	000	1	*	*	001	0011
S_1	001	*	*	0	001	1011
	001	*	*	1	010	1011
S_2	010	*	0	*	010	0011
	010	*	1	0	001	0011
	010	0	1	1	010	0010
	010	1	1	1	011	0010
S_3	011	*	*	0	011	0111
	011	*	*	1	100	0111
S_4	100	*	0	*	101	0111
	100	0	1	*	100	0110
	100	1	1	*	101	0110
S_5	101	0	*	*	101	0001
	101	1	*	*	000	0001
Estados no definidos	110, 111	*	*	*	000	0011

Al tener en cuenta las condiciones de entrada y salida, se puede observar que las codificaciones 110 y 111 no corresponden a un estado de la carta ASM. Para recuperar el funcionamiento ante cualquiera de ellas, las tres entradas se consideran *don't cares* ($X = Y = Z = *$) y la liga se fija en 000. De esta forma sin importar la combinación de entrada, la máquina regresa a S_0 con las salidas inactivas VBPM=0011. Esta condición corresponde a las direcciones 48 a 63 de la Memoria ROM.

2.4. Implementación en VHDL

El funcionamiento de la maquina de estados implementado con una Memoria ROM consta del siguiente flujo. Primero inicia en el registro, donde en cada flanco ascendente de reloj guarda la liga que recibió en el ciclo anterior y la presenta como **edopresente**. En paralelo, el concatenador une los tres bits del estado presente con las entradas XYZ, donde este vector resultante de seis bits selecciona una palabra de la ROM. La palabra leída se divide en dos campos: los bits [6 : 4] forman la liga que volverá al registro en el siguiente flanco, y los bits [3 : 0] se envían de inmediato a las salidas VBPM. Por tanto, el flujo completo del sistema es

registro $\longrightarrow$ concatenador $\longrightarrow$ rom $\longrightarrow$ div_datos $\longrightarrow$ registro.

2.4.1. Componentes del sistema

Cada uno de los bloques que componen el sistema se implementó como un módulo independiente por medio de Lenguaje VHDL

Registro de estado. El módulo **registro** contiene la señal interna **internal_value**, inicializada en 000. Si **reset='0'**, el primer proceso fuerza inmediatamente dicha codificación, por lo que S_0 es el estado seguro de arranque. En cualquier otro caso, el bloque sólo actualiza el estado al detectar **rising_edge(clk)** de esta forma se evita que cambios en las entradas modifiquen el estado entre flancos. El segundo proceso copia el valor interno a **edopresente** para que pueda emplearse como parte de la dirección.

Listing 1: Módulo de registro del estado presente.

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity registro is
7     port(
8         clk : in std_logic;
9         reset : in std_logic;
10        edosiguiente : in std_logic_vector(2 downto 0);
11        edopresente : out std_logic_vector(2 downto 0)
12    );
13 end registro;
14
15 architecture Behavioral of registro is
16
```

```

17  signal internal_value: std_logic_vector(2 downto 0) := B"000";
18
19  begin
20      process(clk, reset, edosiguiente)
21      begin
22          if reset = '0' then
23              internal_value <= B"000";
24          elsif rising_edge(clk) then
25              internal_value <= edosiguiente;
26          end if;
27      end process;
28
29      process(internal_value)
30      begin
31          edopresente <= internal_value;
32      end process;
33
34  end Behavioral;

```

Formación de dirección. El componente de concatenación, cada vez que cambia una entrada o el estado presente, ejecuta `dir <= edopresente & entradas`. Al colocar `edopresente` antes de `entradas`, sus bits son los más significativos. Por ello, todas las direcciones de S_0 comienzan con 000, las de S_1 con 001, y así sucesivamente. Dentro de cada grupo, XYZ selecciona la trayectoria de la carta ASM.

Listing 2: Módulo que concatena el estado presente y las entradas para formar la dirección.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity concatenador is
7      port(
8          entradas: in std_logic_vector(2 downto 0);
9          edopresente: in std_logic_vector(2 downto 0);
10         dir: out std_logic_vector(5 downto 0)
11     );
12 end concatenador;
13
14 architecture Behavioral of concatenador is
15 begin
16     process(entradas, edopresente)
17     begin
18         dir <= edopresente & entradas; -- EdoPresente Bits mas significativos
19     end process;
20 end Behavioral;

```

ROM Para el componente de ROM, el tipo `mem` declara 64 posiciones de siete bits. Cada asignación tiene la forma `"liga"& "salida"`. Donde por ejemplo, las posiciones de S_0 escriben 000&0011 o 001&0011, según X . El proceso de lectura convierte `dir` de binario a entero mediante `conv_integer(unsigned(dir))` y entrega la palabra seleccionada en `data`.

Listing 3: Memoria ROM que codifica las ligas y las salidas.

```

1  library IEEE;

```

```

2 use IEEE.STD_LOGIC_1164.ALL;
3 use IEEE.STD_LOGIC_ARITH.ALL;
4 use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6 entity rom is
7     port(
8         dir: in std_logic_vector(5 downto 0); -- edopresente + entradas
9         data: out std_logic_vector(6 downto 0) -- Liga + Salida
10    );
11 end rom;
12
13 architecture Behavioral of rom is
14     type mem is array (0 to 63) of std_logic_vector(6 downto 0);
15     signal internal_mem: mem;
16
17     begin
18         -- Liga + Salida
19         --S0
20         internal_mem(0) <= "000" & "0011";
21         internal_mem(1) <= "000" & "0011";
22         internal_mem(2) <= "000" & "0011";
23         internal_mem(3) <= "000" & "0011";
24         internal_mem(4) <= "001" & "0011";
25         internal_mem(5) <= "001" & "0011";
26         internal_mem(6) <= "001" & "0011";
27         internal_mem(7) <= "001" & "0011";
28         --S1
29         internal_mem(8) <= "001" & "1011";
30         internal_mem(9) <= "010" & "1011";
31         internal_mem(10) <= "001" & "1011";
32         internal_mem(11) <= "010" & "1011";
33         internal_mem(12) <= "001" & "1011";
34         internal_mem(13) <= "010" & "1011";
35         internal_mem(14) <= "001" & "1011";
36         internal_mem(15) <= "010" & "1011";
37         --S2
38         internal_mem(16) <= "010" & "0011";
39         internal_mem(17) <= "010" & "0011";
40         internal_mem(18) <= "001" & "0011";
41         internal_mem(19) <= "010" & "0010";
42         internal_mem(20) <= "010" & "0011";
43         internal_mem(21) <= "010" & "0011";
44         internal_mem(22) <= "001" & "0011";
45         internal_mem(23) <= "011" & "0010";
46         --S3
47         internal_mem(24) <= "011" & "0111";
48         internal_mem(25) <= "100" & "0111";
49         internal_mem(26) <= "011" & "0111";
50         internal_mem(27) <= "100" & "0111";
51         internal_mem(28) <= "011" & "0111";
52         internal_mem(29) <= "100" & "0111";
53         internal_mem(30) <= "011" & "0111";
54         internal_mem(31) <= "100" & "0111";
55         --S4
56         internal_mem(32) <= "101" & "0111";
57         internal_mem(33) <= "101" & "0111";
58         internal_mem(34) <= "100" & "0110";
59         internal_mem(35) <= "100" & "0110";
60         internal_mem(36) <= "101" & "0111";
61         internal_mem(37) <= "101" & "0111";
62         internal_mem(38) <= "101" & "0110";
63         internal_mem(39) <= "101" & "0110";
64         --S5

```

```

65     internal_mem(40) <= "101" & "0001";
66     internal_mem(41) <= "101" & "0001";
67     internal_mem(42) <= "101" & "0001";
68     internal_mem(43) <= "101" & "0001";
69
70     internal_mem(44) <= "000" & "0001";
71     internal_mem(45) <= "000" & "0001";
72     internal_mem(46) <= "000" & "0001";
73     internal_mem(47) <= "000" & "0001";
74
75     process(dir)
76     begin
77         data <= internal_mem(conv_integer(unsigned(dir)));
78     end process;
79 end Behavioral;

```

División de datos. Posteriormente, el componente `div_datos` toma `data(6 downto 4)` como liga y `data(3 downto 0)` como VBPM, esto lo realiza por cada que la memoria ROM entrega una palabra. De esta forma, el registro recibe la liga y las salidas se actualizan de inmediato.

Listing 4: Módulo que separa liga y salidas de la palabra de la ROM.

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity div_datos is
7      port(
8          data: in std_logic_vector(6 downto 0);
9          liga: out std_logic_vector(2 downto 0);
10         salidas: out std_logic_vector(3 downto 0)
11     );
12 end div_datos;
13
14 architecture Behavioral of div_datos is
15 begin
16     process(data)
17     begin
18         liga <= data(6 downto 4);
19         salidas <= data(3 downto 0);
20     end process;
21 end Behavioral;

```

Acondicionamiento del botón. Para el control de la señal de reloj, se reutilizó el módulo `clk_button` de la práctica anterior. Este bloque genera un pulso de reloj por cada pulsación del botón, y además filtra el rebote mecánico que se produce al presionar el interruptor.

3. Simulaciones

A partir de las simulaciones se verificó el recorrido de los seis estados, así como cada combinación `Q2Q1Q0XYZ` y el contenido `L2L1L0VBPM` de la ROM. Para permitir el funcionamiento

normal del sistema, **reset** se mantiene en 1 durante la simulación ya que la señal de entrada de reset se activa con un 0. Las entradas se probaron primero con combinaciones aleatorias y, en el último tramo, se forzaron a 111 para recorrer de forma visible todos los estados de la carta ASM. En las gráficas obtenidas, las señales de entrada aparecen como A, B y C estas corresponden a las entradas X, Y y Z utilizadas en el diseño.

En Figura[2] se presenta la vista general del sistema. Donde al inicio, **reset=1** permite que el sistema opere normalmente y el estado presente **edo_p** parte de 000, correspondiente al estado S_0 . El reloj establece los instantes en los que el registro puede aceptar la liga, mientras que las entradas cambian en distintos intervalos para probar varias localidades de la ROM. En cada instante se forma la dirección Q2Q1Q0XYZ, donde los tres bits de **edo_p** son los bits más significativos y las entradas seleccionan la trayectoria dentro del estado actual.

La ROM entrega una palabra de siete bits. Sus bits L2L1L0 representan la liga que se almacenará como estado siguiente y sus bits VBPM representan las salidas. Se puede observar que al tener salidas condicionales, un cambio en las entradas puede afectar la salida del sistema, pero **edo_p** sólo cambia en el siguiente flanco ascendente de **clk**. Este comportamiento permite distinguir en la forma de onda la respuesta combinacional de la ROM de la actualización secuencial del registro.

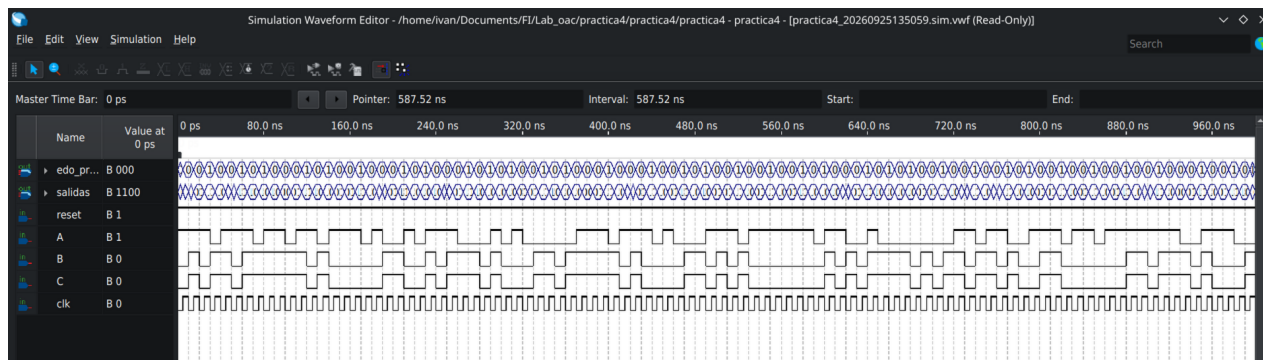


Figura 2: Primera sección de la simulación. Se mantienen **reset=1** y combinaciones aleatorias en **entradas**.

La Figura[3] se muestra el comportamiento anteriormente descrito. En esta gráfica se visualizan los cambios de A, B y C y su relación con los valores de **edo_p**. Por ejemplo, cuando la combinación de entradas selecciona una salida condicional que conserva el estado, el valor de **edo_p** se repite después del flanco cuando selecciona otra trayectoria, sin embargo aparece la nueva codificación de estado en el siguiente ciclo. Es importante considerar que para las señales de salidas V y B usan lógica positiva, mientras que P y M son activos en bajo, por lo que un cero en estos últimos indica que la función está activa.

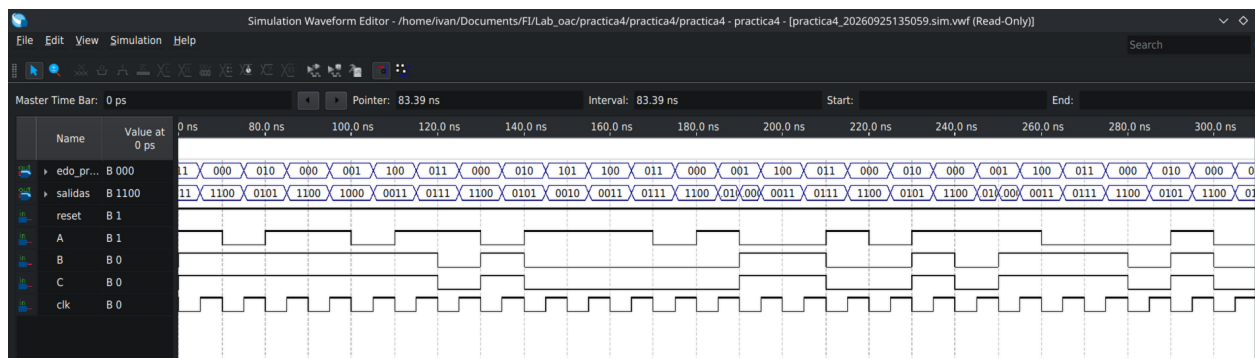


Figura 3: Detalle de la respuesta de `edo_p` y VBPM ante cambios aleatorios en las entradas.

La Figura[4] corresponde al final de la simulación, donde las entradas se fuerzan a 111. Esta condición facilita permite que la carta ASM transicione por todos los estados. La secuencia de `edo_p` permite observar el avance por los estados válidos y, al llegar a un estado cuya transición depende de las entradas, la liga almacenada en la ROM determina el siguiente estado. Por lo que se comprueba el ciclo completo de la máquina y se verifica que el diseño implementado en VHDL cumple con la carta ASM de la lavadora industrial.

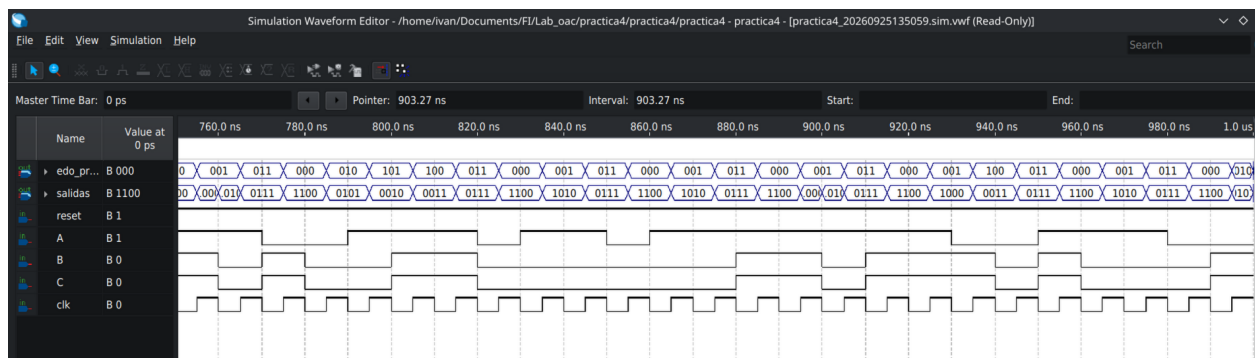


Figura 4: Tramo final forzado con `entradas=111`; se observa el recorrido completo por los estados de la carta ASM.

4. Resultados

Las Figuras[5] y [6] muestran el diagrama de bloques implementado en Quartus, el cual implementa el flujo de los componentes previamente descritos. Donde el reloj y el botón ingresan al bloque `clk_button`, el registro entrega el estado presente, el concatenador genera la dirección de seis bits y la ROM produce la palabra de siete bits. El bloque `div_datos` distribuye la liga hacia el registro y las salidas VBPM hacia los pines de la tarjeta.

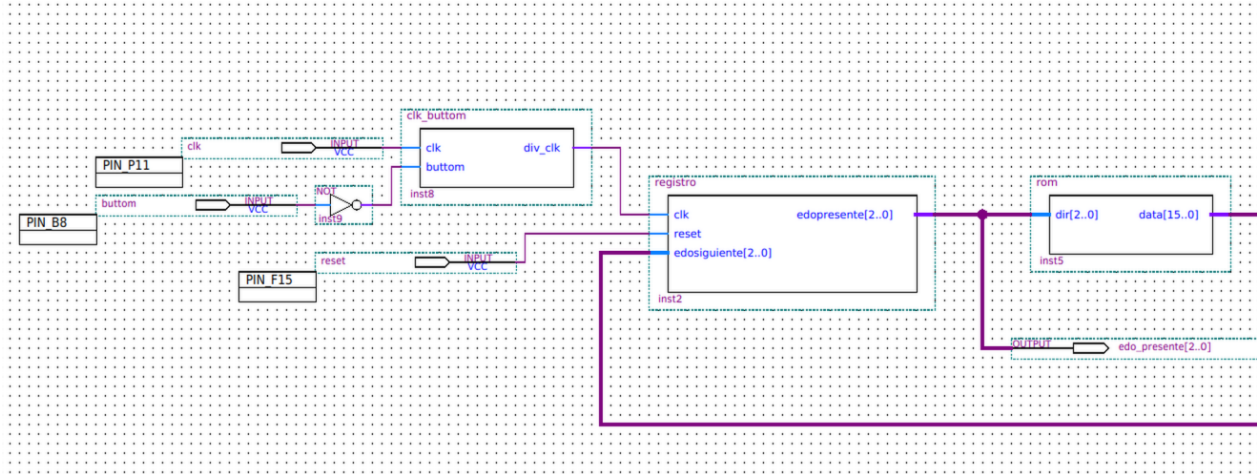


Figura 5: Interconexión del reloj, el registro de estado, las entradas y el generador de dirección.

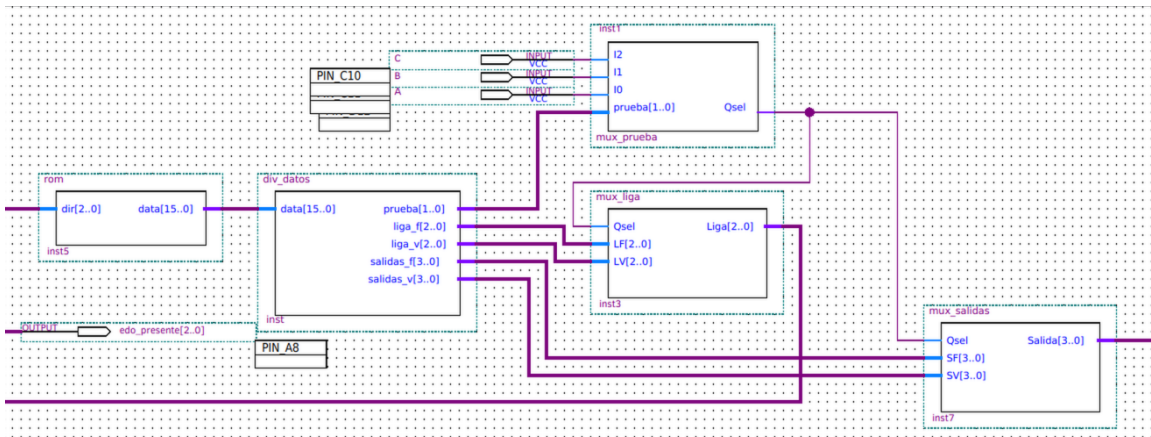
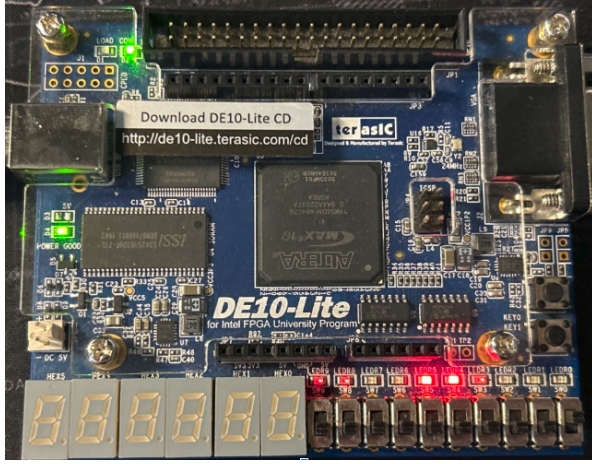


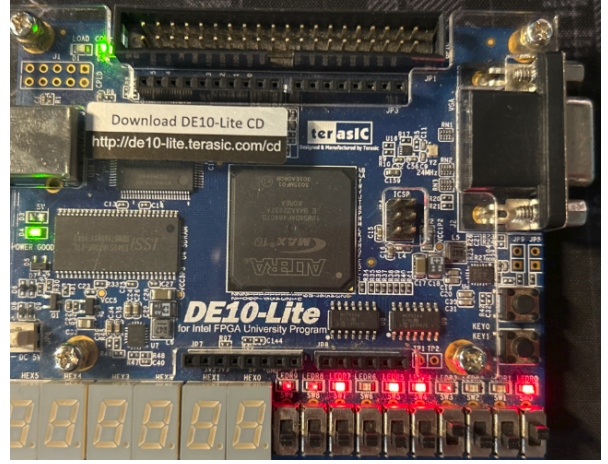
Figura 6: ROM y división de la palabra de datos en liga y salidas VBPM.

4.1. Evidencia en la tarjeta FPGA

Al cargar el diseño en la tarjeta FPGA, se verificó el funcionamiento de la máquina de estados mediante los LEDs asociados al estado presente **edo_p** y a las salidas VBPM. Para cada evidencia se seleccionó la combinación de entradas indicada en la tabla de transición y se comparó la respuesta observada con la liga y las salidas almacenadas en la ROM.

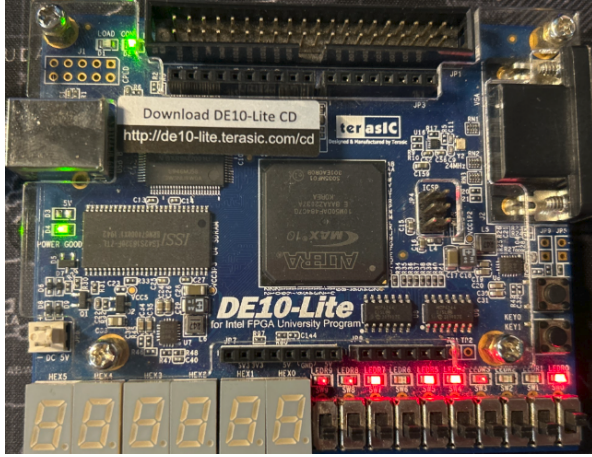


(a) $S_0, X = 0$: se conserva el estado inicial.

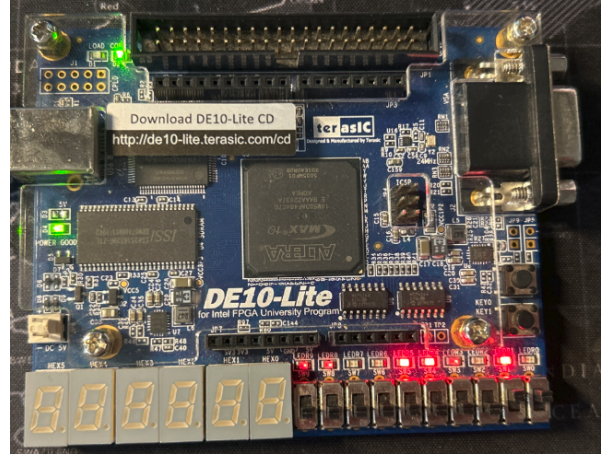


(b) $S_0, X = 1$: liga hacia S_1 .

Figura 7: Evidencia en tarjeta para las trayectorias del estado S_0 .

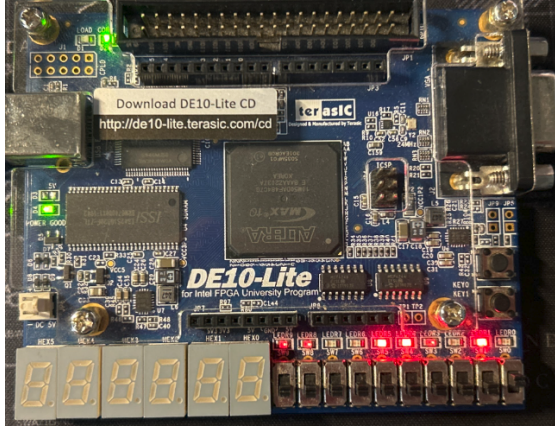


(a) $S_1, Z = 0$: permanece en llenado.

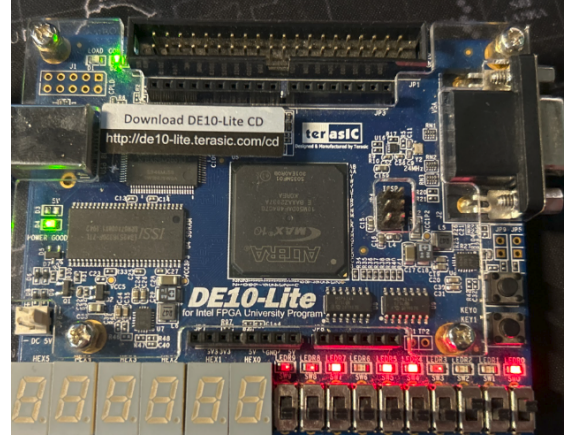


(b) $S_1, Z = 1$: liga hacia S_2 .

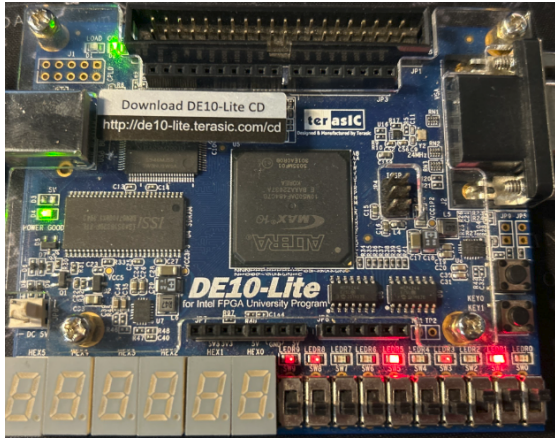
Figura 8: Evidencia en tarjeta para las trayectorias del estado S_1 .



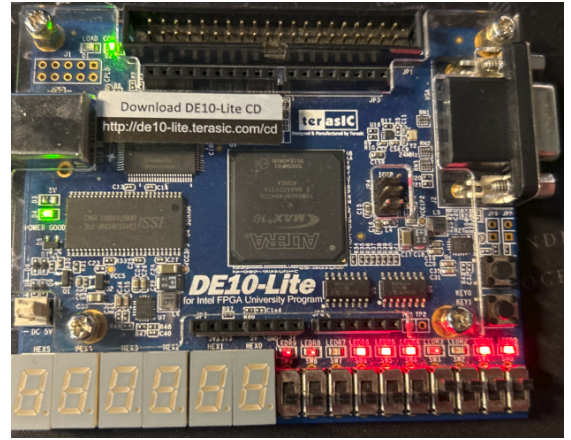
(a) S_2 , $Y = 0$.



(b) S_2 , $Y = 1$, $Z = 0$: liga hacia S_1 .

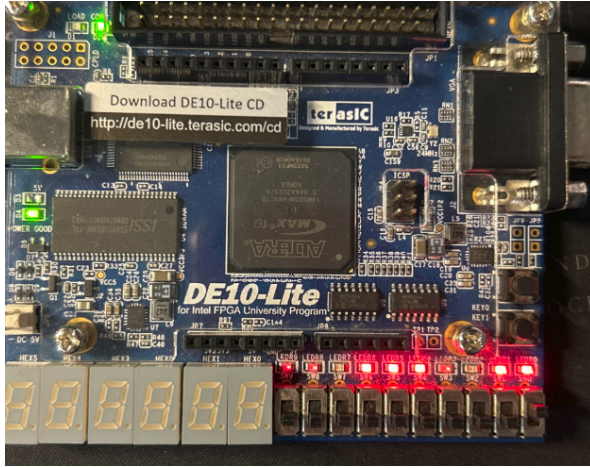


(c) S_2 , $X = 0$, $Y = 1$, $Z = 1$.

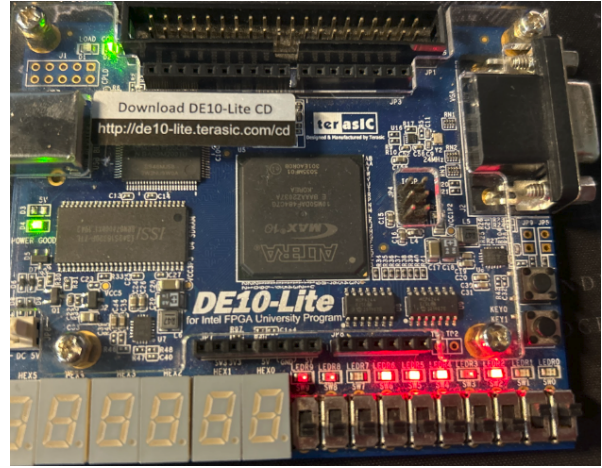


(d) S_2 , $X = 1$, $Y = 1$, $Z = 1$: liga hacia S_3 .

Figura 9: Evidencia en tarjeta para las cuatro trayectorias del estado S_2 .

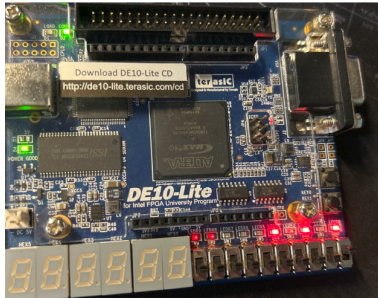


(a) $S_3, Z = 0$: permanece en drenaje.

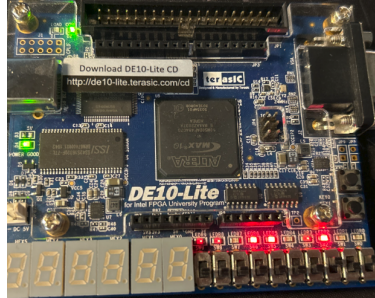


(b) $S_3, Z = 1$: liga hacia S_4 .

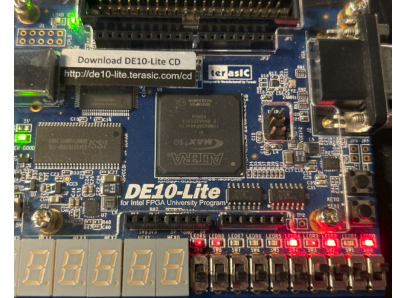
Figura 10: Evidencia en tarjeta para las trayectorias del estado S_3 .



(a) $S_4, Y = 0$.

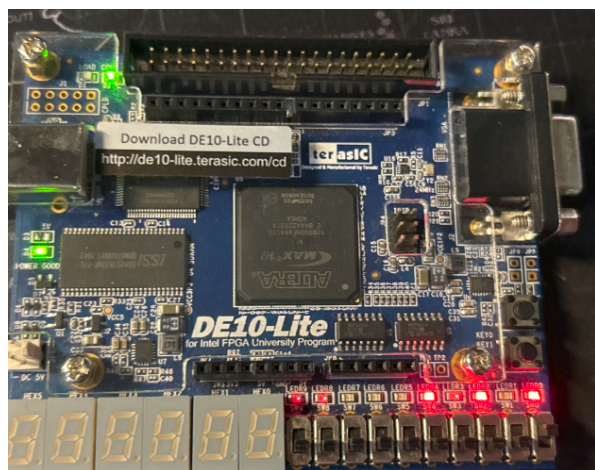


(b) $S_4, X = 0, Y = 1$.

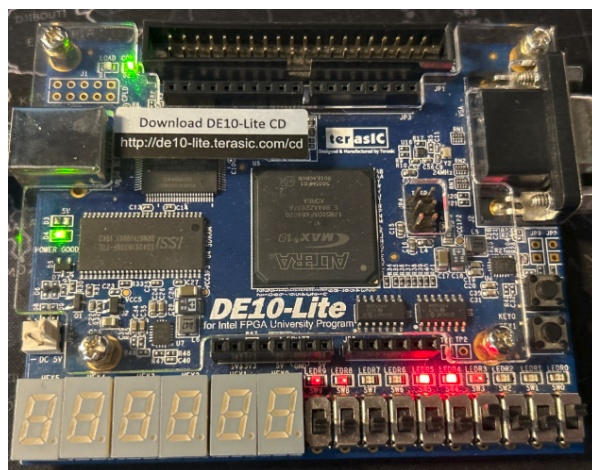


(c) $S_4, X = 1, Y = 1$.

Figura 11: Evidencia en tarjeta para las trayectorias del estado S_4 ; Z es indiferente en este estado.



(a) S_5 , $X = 0$: mantiene la alarma.



(b) S_5 , $X = 1$: liga hacia S_0 .

Figura 12: Evidencia en tarjeta para las trayectorias del estado S_5 .

5. Conclusión

Percival Ulises Espinoza Matamoros. Gracias a la realización de la práctica pude comprender la importancia del direccionamiento por trayectoria, ya que al implementar la carta ASM por medio de una memoria ROM junto a otros componentes, donde en la memoria ROM se almacenan las ligas y salidas correspondientes a cada combinación de estado presente y entradas, donde pude observar cómo las entradas determinan la liga y las salidas en cada ciclo de reloj. Al verificar el funcionamiento por medio de la simulación y la implementación física en la tarjeta FPGA, se pudo tener un mejor entendimiento de como funciona el diseño implementado.

Oscar Iván Montiel Juárez. En esta práctica aplicamos conceptos sobre cartas ASM, codificación de estados y memorias ROM. Al llevarlos a VHDL y a la tarjeta FPGA, se pudo relacionar la tabla de direccionamiento con el funcionamiento real del circuito, observando cómo las entradas determinan la liga y las salidas en cada ciclo de reloj. Es decir podemos afirmar que hemos comprendido el funcionamiento del direccionamiento por trayectoria (tanto la parte práctica como teórica) y tenemos evidencia que respalda esta afirmación, además pudimos construir en diagrama de bloques .

Amy Valentina Veraza García. El desarrollo mostró la importancia de validar el diseño en más de una etapa. La simulación permitió revisar las transiciones antes de implementarlas, mientras que las pruebas en la tarjeta confirmaron que el registro de estado, la ROM y las salidas responden de acuerdo con la secuencia definida para la llenadora industrial.